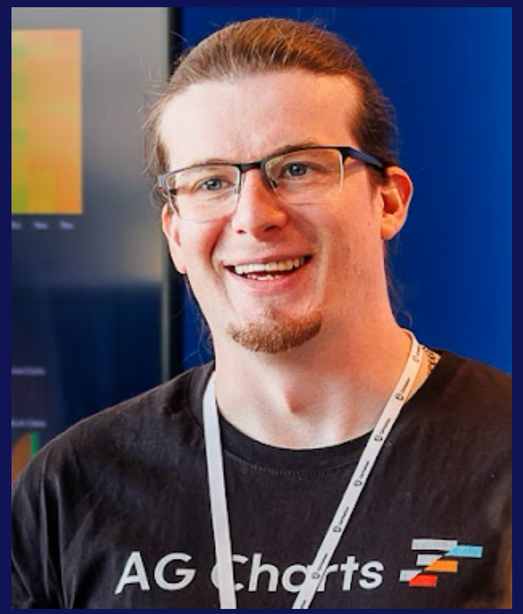
Build a payments dashboard without building a dashboard

Eddie Jaoude & James Swinton-Bland

PayPal Hackathon, Oct - Nov, 2026

PayPal Developer

Speakers



James Swinton-Bland

DevRel Lead @ AG Grid

Eddie Jaoude

DevRel @ PayPal

AG Grid × PayPal Developer

Agenda

01 Live Demo

02 PayPal APIs

03 Dashboard Architecture

04 APIs -> AG Studio

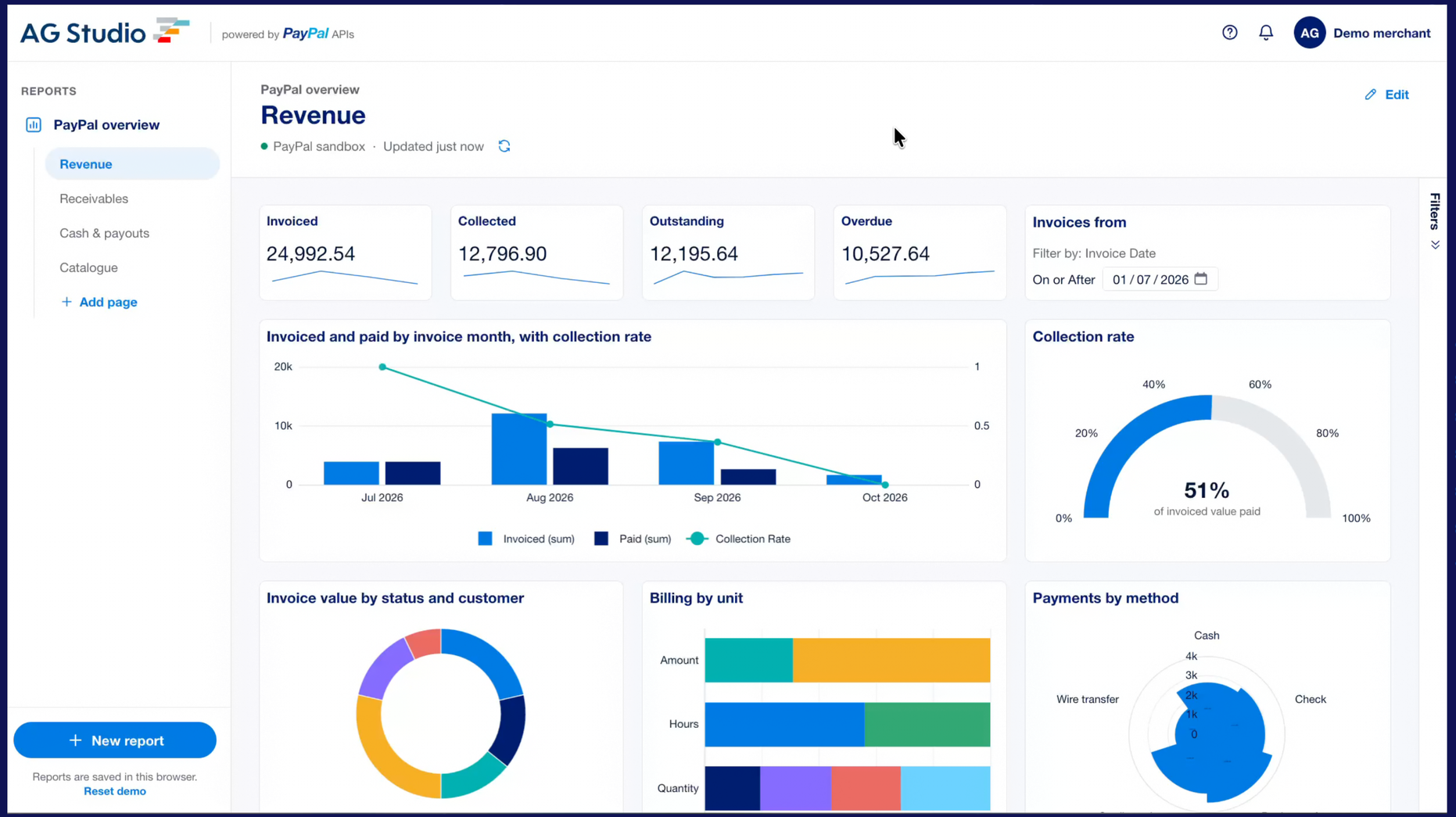
05 Customising AG Studio

06 Connecting Studio to your UI

AG Grid × PayPal Developer

01

Live Demo



AG Grid × PayPal Developer

AG Studio | powered by PayPal APIs

AG Demo merchant

REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

+ Add page

+ New report

Reports are saved in this browser.
Reset demo

PayPal overview

Revenue

PayPal sandbox · Updated just now

Edit

Filters

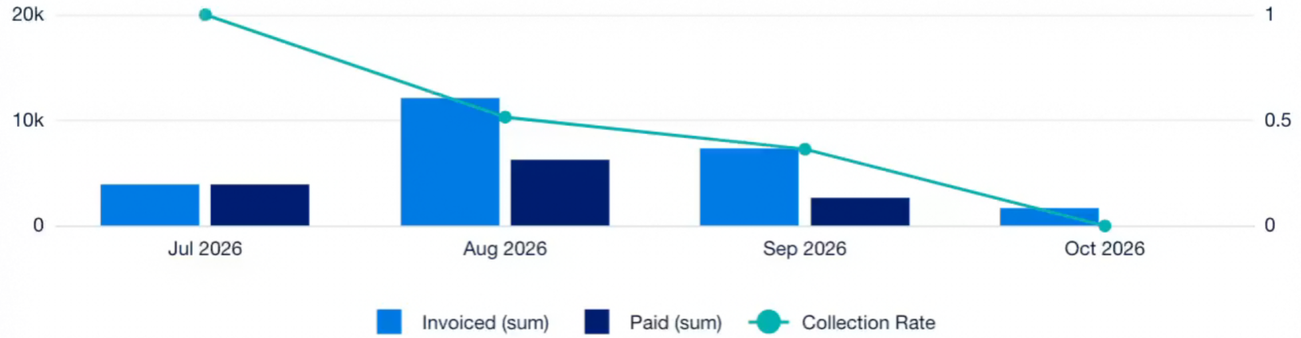
Invoiced	Collected	Outstanding	Overdue
24,992.54	12,796.90	12,195.64	10,527.64

Invoices from

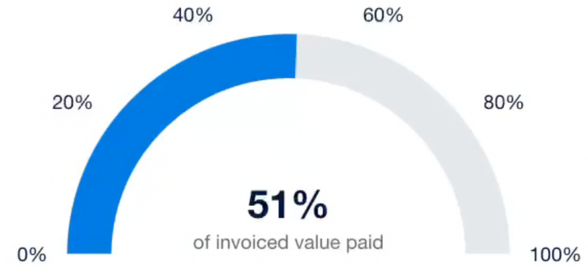
Filter by: Invoice Date

On or After 01 / 07 / 2026

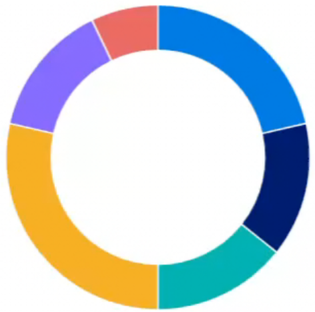
Invoiced and paid by invoice month, with collection rate



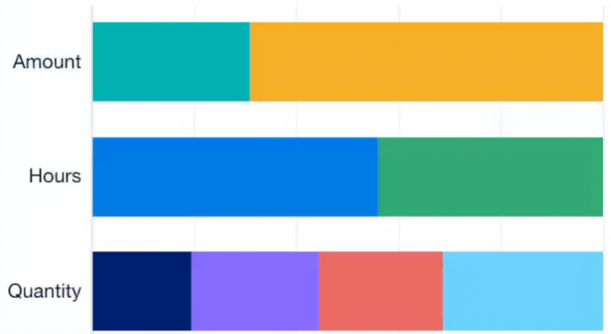
Collection rate



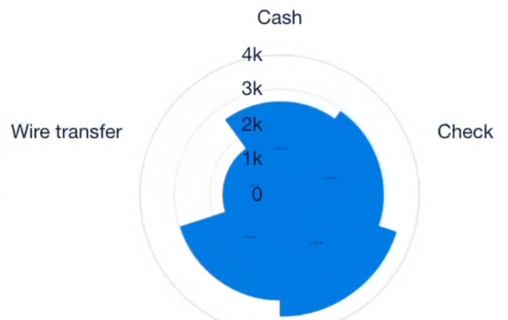
Invoice value by status and customer



Billing by unit



Payments by method



REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

+ Add page

PayPal overview

Revenue

PayPal sandbox · Updated just now

Edit

Filters

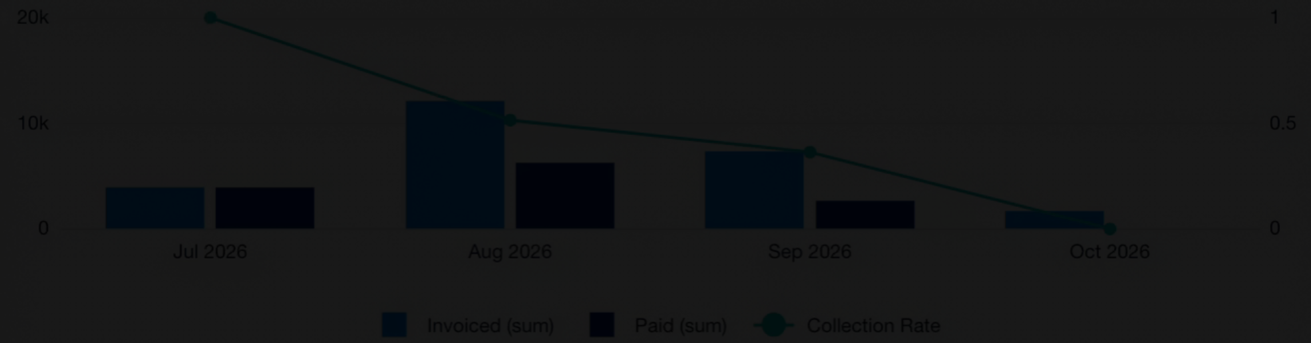
Invoiced	Collected	Outstanding	Overdue
24,992.54	12,796.90	12,195.64	10,527.64

Invoices from

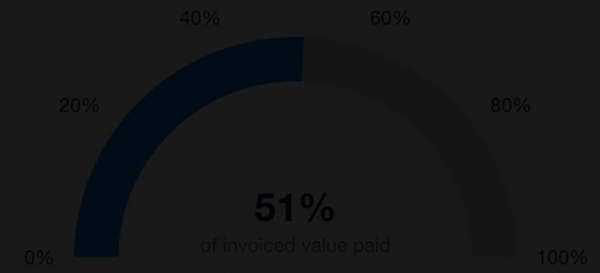
Filter by: Invoice Date

On or After 01 / 07 / 2026

Invoiced and paid by invoice month, with collection rate

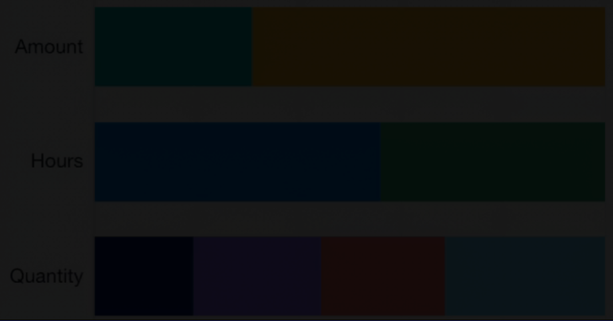


Collection rate

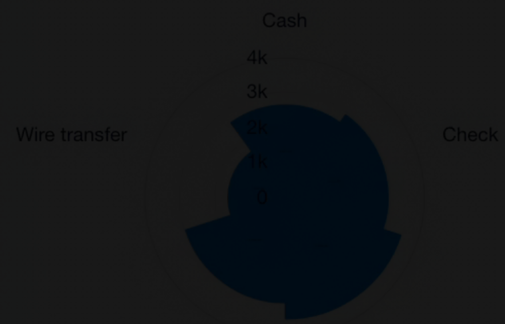


Invoice value by status and customer

Billing by unit



Payments by method



+ New report

Reports are saved in this browser.

Reset demo

AG Studio powered by PayPal APIs

Demo merchant

REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

Add page

New report

Reports are saved in this browser.
Reset demo

PayPal overview

Revenue

PayPal sandbox · Updated just now

Edit

Filters

Invoiced
24,992.54

Collected
12,796.90

Outstanding
12,195.64

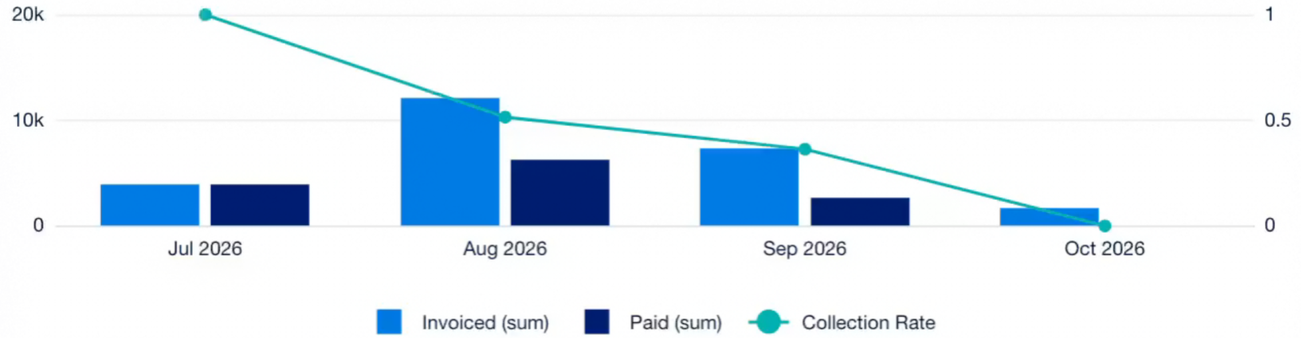
Overdue
10,527.64

Invoices from

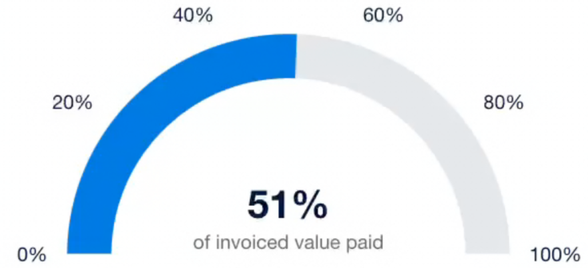
Filter by: Invoice Date

On or After 01 / 07 / 2026

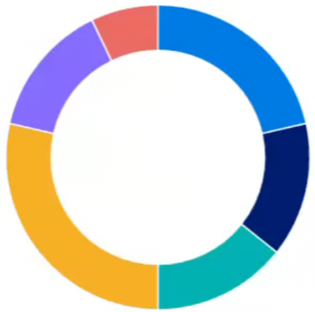
Invoiced and paid by invoice month, with collection rate



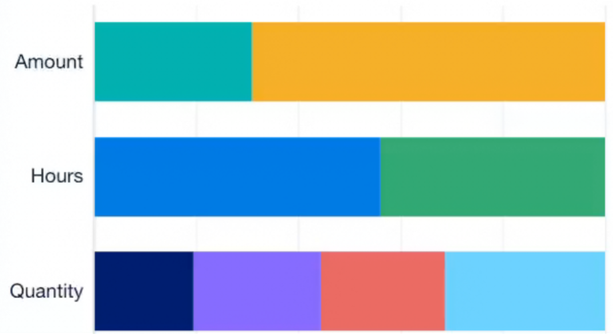
Collection rate



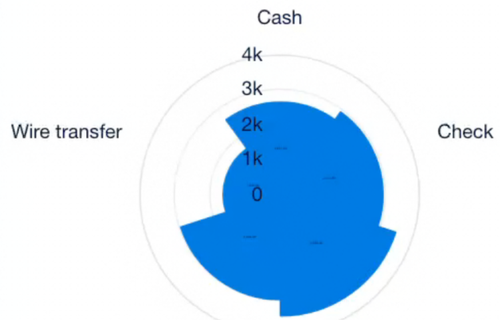
Invoice value by status and customer



Billing by unit



Payments by method



8

REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

+ Add page

PayPal overview

Revenue

PayPal sandbox · Updated just now

Edit

Filters

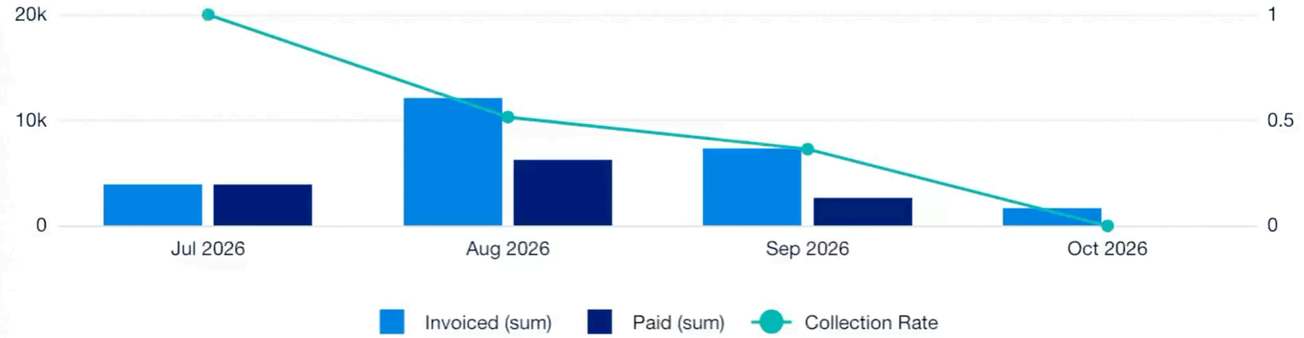
Invoiced	Collected	Outstanding	Overdue
24,992.54	12,796.90	12,195.64	10,527.64

Invoices from

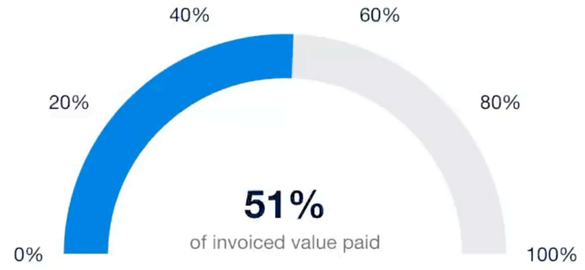
Filter by: Invoice Date

On or After 01 / 07 / 2026

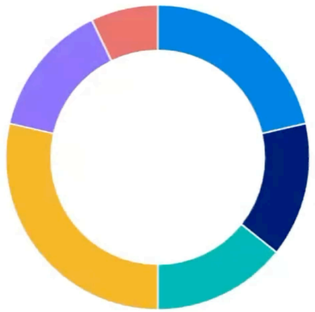
Invoiced and paid by invoice month, with collection rate



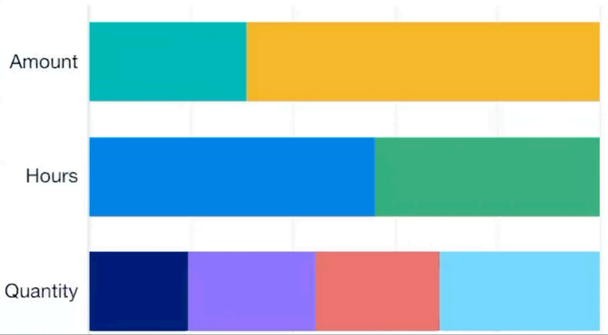
Collection rate



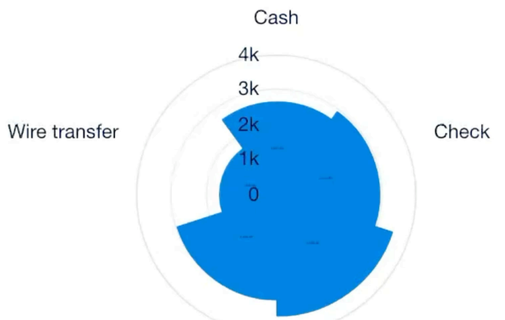
Invoice value by status and customer



Billing by unit



Payments by method



+ New report

Reports are saved in this browser.
Reset demo

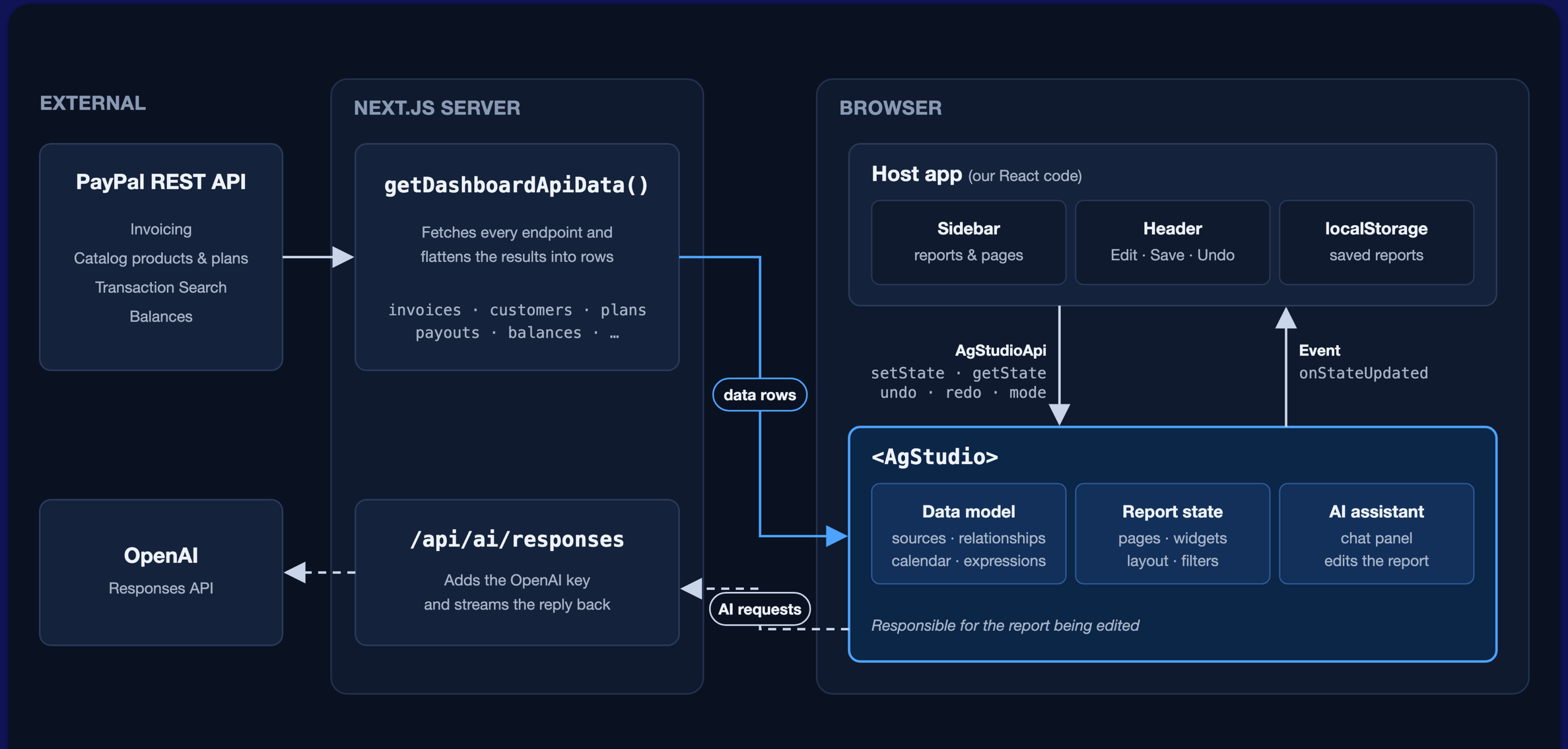
02

PayPal
+ Sandbox
+ APIs

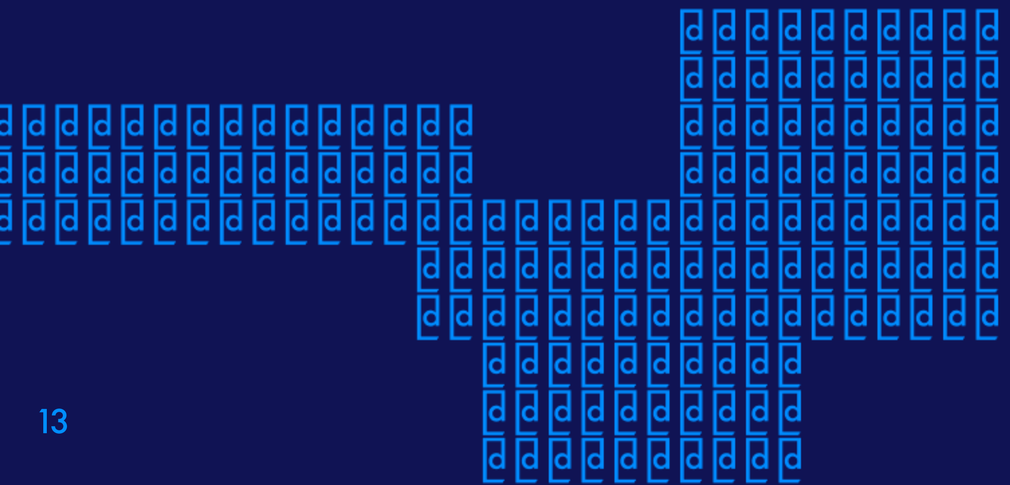
03

Dashboard Architecture

Architecture



Source Code



04

APIs => AG Studio

Loading data

Page.tsx

```tsx
/*
* Calls the PayPal REST API, and passes the flat list of data to the
* client-side Dashboard component which includes AG Studio.
* /

import { connection } from 'next/server';
import { getDashboardApiData } from '@/lib/dashboard-api';
import { Dashboard } from '@/app/dashboard-api/dashboard';

export default async function DashboardPage() {
  await connection();

  // Calls the PayPal REST API, then turns every response into flat rows
  const data = await getDashboardApiData();
  // data = { invoices: [...], customers: [...], plans: [...], payouts: [...], ... }

  // Pass the data to the client component, which renders the dashboard
  return <Dashboard data={data} licenseKey={process.env.AG_STUDIO} />;
}
```

Defining Data

Data.ts

```
/*
* Function to convert raw API data into the format AG Studio expects. Each REST endpoint
* becomes a table in AG Studio, with definitions for each of the fields within the
* response.
* /

export function buildStudioData(data: DashboardApiData): AgDataSourcesDefinition {
  return {
    description: 'PayPal sandbox data for a merchant selling subscription products’,
    sources: [
      {
        id: 'invoices',
        description: 'One row per invoice.’,
        data: data.invoices,
        fields: [
          { id: 'id', name: 'Invoice ID', format: 'textFormat', hide: true },
          { id: 'customerId', name: 'Customer ID', format: 'textFormat', hide: true },
         // ...additional fields
        ],
      },
      // ...additional sources
    ],
  };
}
```

Defining Relationships

Data.ts

```ts
/*
* Relationships allow you to execute queries across different data sources (e.g.
* different PayPal APIs). Relationships work in the same way as relational databases.
* /

import type { AgDataSourcesDefinition } from 'ag-studio';
import type { DashboardApiData } from '@/lib/dashboard-api';

export function buildStudioData(data: DashboardApiData): AgDataSourcesDefinition {
  return {
    sources: [ /* ... */ ],
    relationships: [
      // Each invoice belongs to a single customer
      {
        id: 'invoice-customer',
        type: 'many-to-one',
        source: { tableId: 'invoices', fieldId: 'customerId' },
        target: { tableId: 'customers', fieldId: 'id' },
      },
      // ...additional relationships
    ],
  };
}
```

Creating new Fields

Data.ts

```
// Expressions allow you to calculate new fields from your existing data.

export function buildStudioData(data: DashboardApiData): AgDataSourcesDefinition {
  return {
    sources: [ /* ... */ ],
    relationships: [ /* ... */ ],
    expressions: [
      {
        id: 'collectionRate',
        description: 'Paid / invoiced. Filter to one currency first.',
        format: 'percentageFormat',
        isMeasure: true, // true = single value, false = column
        expression: {
          operator: 'divide',
          inputs: [
            { id: 'invoices.paidAmount', aggregation: 'sum' },
            { id: 'invoices.issuedAmount', aggregation: 'sum' },
          ],
        },
      },
    ],
  };
}
```

Managing Date Ranges

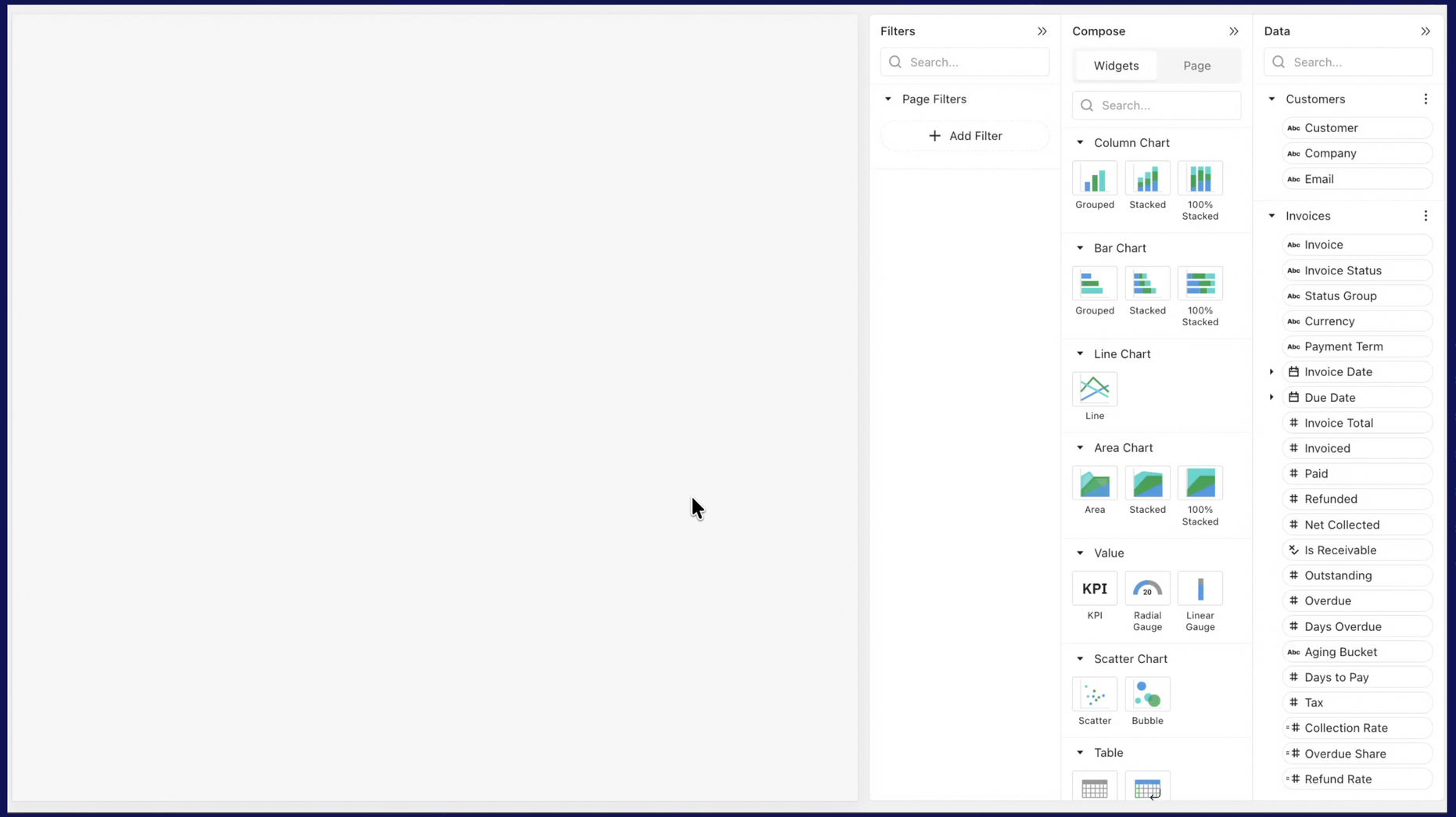
Data.ts

```
// Calendars allows you to define groups for your dates, e.g. years, weeks, months, etc..

export function buildStudioData(data: DashboardApiData): AgDataSourcesDefinition {
  // Every date in the data, so the calendar covers exactly that range
  const dates = getDateRange(data);

  return {
    sources: [ /* ... */ ],
    relationships: [ /* ... */ ],
    expressions: [ /* ... */ ],
    calendars: [
      {
        id: 'calendar',
        label: 'Calendar',
        range: {
          from: { type: 'date', value: dates.start },
          to: { type: 'date', value: dates.end },
        },
        fragments: ['year', 'quarter', 'month', 'week', 'day', 'dayOfWeek'],
      },
    ],
  };
}
```

Passing data to AG Studio

Dashboard.tsx

```tsx
/*
* Takes the data from the Page.tsx, builds this into the format that AG Studio
* expects, and passes it to the AgStudio component, along with the license key.
* /

'use client';

import { AgStudio, AgStudioProvider } from 'ag-studio-react';
import type { DashboardApiData } from '@/lib/dashboard-api';
import { buildStudioData } from '@/.../data';

export function Dashboard({ data, licenseKey }) {
  const studioData = useMemo(() => buildStudioData(data), [data]);

  return (
    <AgStudioProvider licenseKey={licenseKey}>
      <AgStudio
        data={studioData}
      />
    </AgStudioProvider>
  );
}
```



AG Grid × PayPal Developer

Filters

Search...

Page Filters

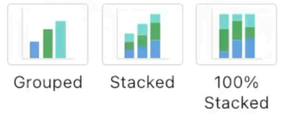
+ Add Filter

Compose

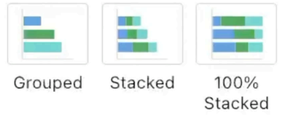
Widgets | Page

Search...

Column Chart

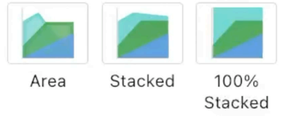


Bar Chart

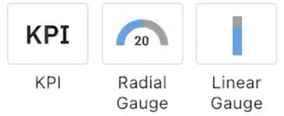


Line Chart

Area Chart



Value



Scatter Chart

Table

Data

Search...

Customers

- Abc Customer
- Abc Company
- Abc Email

Invoices

- Abc Invoice
- Abc Invoice Status
- Abc Status Group
- Abc Currency
- Abc Payment Term
- Invoice Date
- Due Date
- # Invoice Total
- # Invoiced
- # Paid
- # Refunded
- # Net Collected
- Is Receivable
- # Outstanding
- # Overdue
- # Days Overdue
- Abc Aging Bucket
- # Days to Pay
- # Tax
- # Collection Rate
- # Overdue Share
- # Refund Rate

05

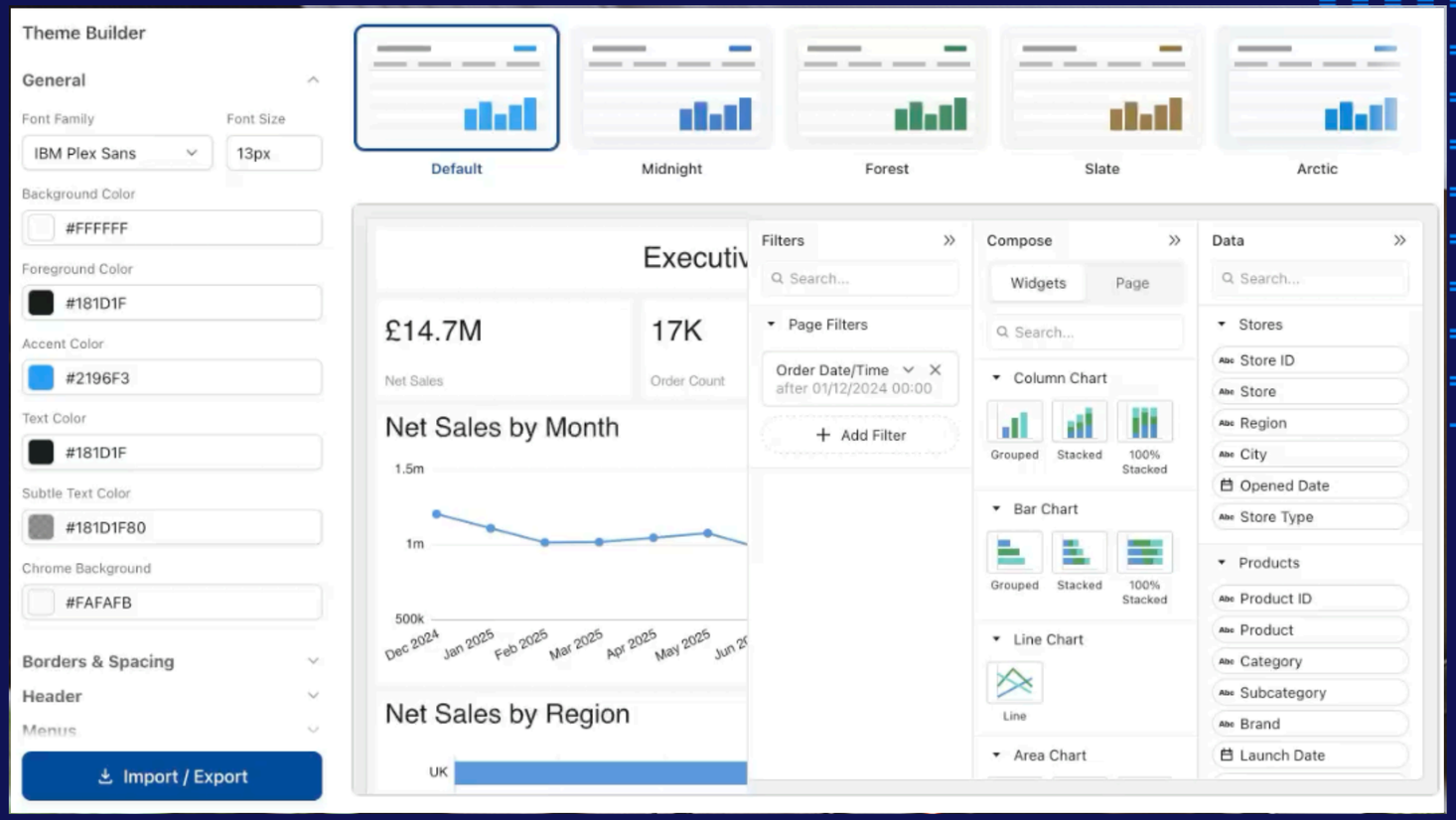
Customising AG Studio

Building a Theme

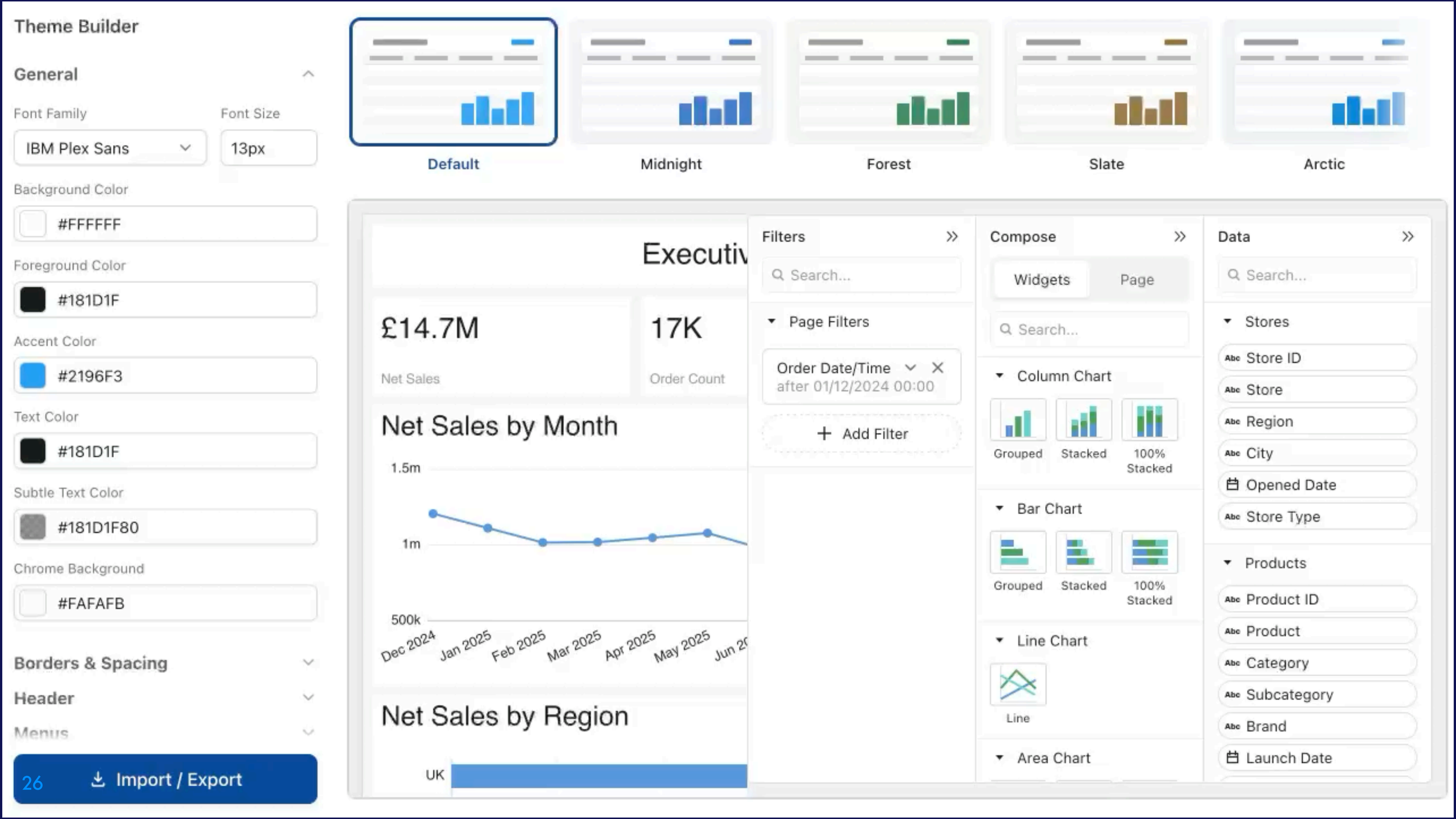
ag-studio-paypal-theme.ts

```
// TODO

import { studioTheme } from 'ag-studio';

export const agStudioPayPalTheme = studioTheme.withParams({
  // Shared by Studio's own UI, grids and charts
  fontFamily: 'var(--pp-font)',
  textColor: 'var(--pp-ink)',
  backgroundColor: 'var(--pp-surface)',
  borderColor: 'var(--pp-border)',
  accentColor: 'var(--pp-blue)',

  // Grids and charts
  gridHeaderTextColor: 'var(--pp-muted)',
  gridRowHeight: 40,
  chartPaletteFills1Color: 'var(--pp-chart-1)',
  chartPaletteFills2Color: ‘var(--pp-chart-2)',

  // ...other theme parameters
});
```

AG Studio Theme Builder



AG Grid × PayPal Developer


Theme Builder
General
Font Family
IBM Plex Sans
Font Size
13px
Background Color
#FFFFFF
Foreground Color
#181D1F
Accent Color
#2196F3
Text Color
#181D1F
Subtle Text Color
#181D1F80
Chrome Background
#FAFAFB
Borders & Spacing
Header
Menus
26
Import / Export
Default
Midnight
Forest
Slate
Arctic
£14.7M
Net Sales
17K
Order Count
Net Sales by Month
1.5m
1m
500k
Dec 2024
Jan 2025
Feb 2025
Mar 2025
Apr 2025
May 2025
Net Sales by Region
UK
Filters
Search...
Page Filters
Order Date/Time
after 01/12/2024 00:00
Add Filter
Compose
Widgets
Page
Search...
Column Chart
Grouped
Stacked
100% Stacked
Bar Chart
Grouped
Stacked
100% Stacked
Line Chart
Line
Area Chart
Data
Search...
Stores
Store ID
Store
Region
City
Opened Date
Store Type
Products
Product ID
Product
Category
Subcategory
Brand
Launch Date

Pre-building Reports

Example.ts

```ts
/*
* Every report is stored within an AgReportState object, with a list of Pages
* (AgPageState) that define the individual reports within Studio.

import type { AgPageState, AgReportState } from 'ag-studio';

const agStudioPage: AgPageState = {
  id: ‘page-id',
  widgets: { ... },
  widgetLayout: { ... },
  filter: { ... },
};

export const dashboardReport: AgReportState = {
  selectedPageId: 'revenue',
  pages: [ agStudioPage ],
};
```

Pre-building Reports

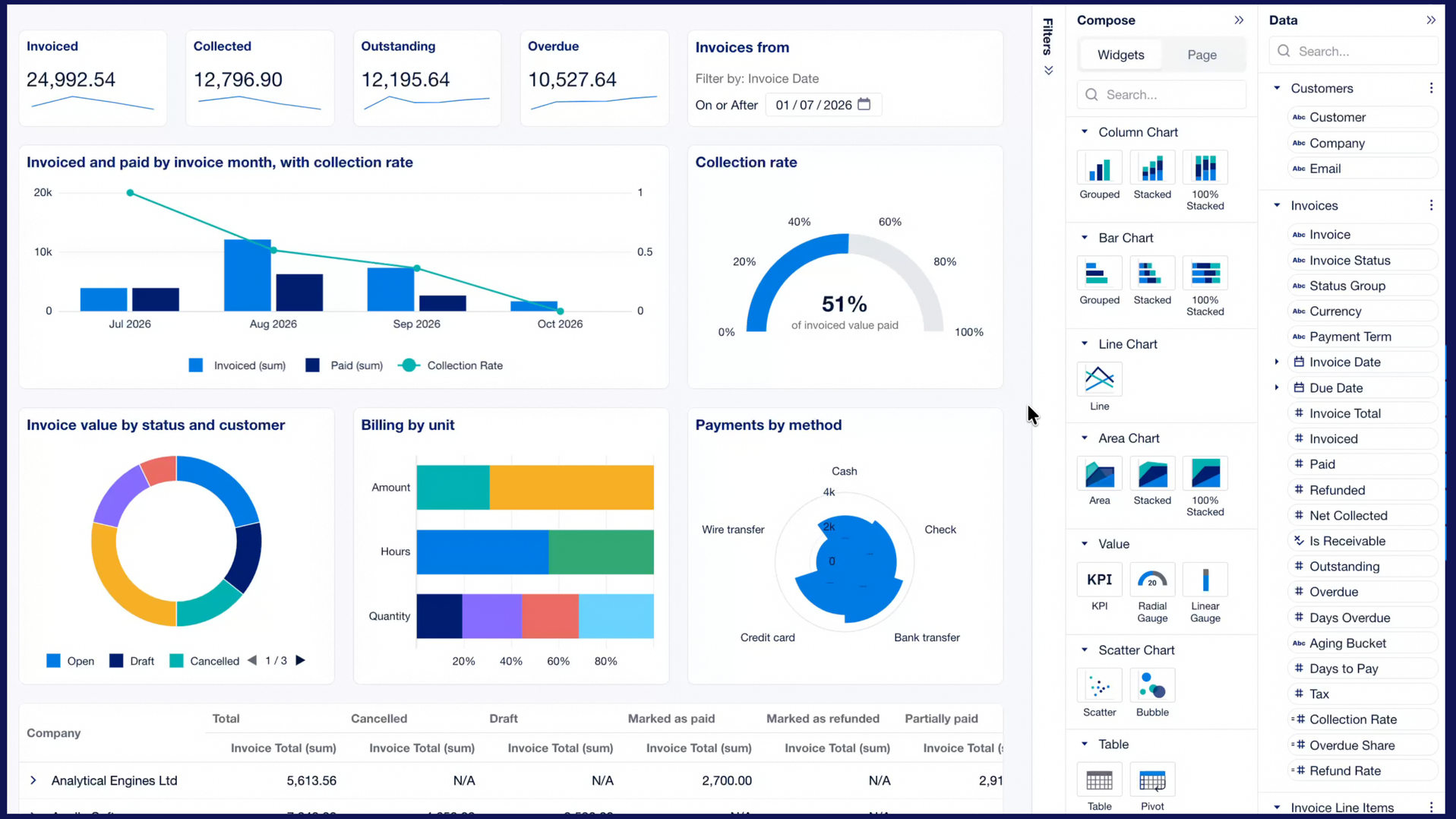
InitialState.ts

```
// This example demonstrates a single report, with a single widget and page-level filter

const revenue: AgPageState = {
  id: ‘revenue',
  widgets: {
   'open-invoices': {
      type: 'grid',
      dataMapping: {
        cols: [
          { id: 'invoices.number' },
        ],
      },
      sort: [{ field: { id: 'invoices.daysOverdue' }, direction: 'desc' }],
      format: { title: { enabled: true, text: 'Open invoices, most overdue first' } },
    },
    // ...More widgets
  },
  widgetLayout: {
    'open-invoices': { xTrack: 0, yTrack: 25, xSpan: 24, ySpan: 20 },
    // …More widgets
  },
  filter: {
    page: [
     { field: { id: 'invoices.isReceivable' }, model: { operator: 'isTrue' } },
    ],
  },
};

export const dashboardState = { selectedPageId: ‘revenue', pages: [ revenue, /* ... */ ] };
```

Defining an Initial State

Dashboard.ts

```
/*
* Takes the data from the Page.tsx, builds this into the format that AG Studio
* expects, and passes it to the AgStudio component, along with the license key.
* /

'use client';

import { AgStudio, AgStudioProvider } from 'ag-studio-react';
import type { DashboardApiData } from '@/lib/dashboard-api';
import { buildStudioData } from '@/.../data';
import { dashboardState } from '@/.../config/initialState';

export function Dashboard({ data, licenseKey }) {
 const studioData = useMemo(() => buildStudioData(data), [data]);
 const [initialState] = useState(dashboardReport);

  return (
    <AgStudioProvider licenseKey={licenseKey}>
      <AgStudio
        data={studioData} initialState={dashboardReport}
      />
    </AgStudioProvider>
  );
}
```



AG Grid × PayPal Developer

Invoiced
24,992.54

Collected
12,796.90

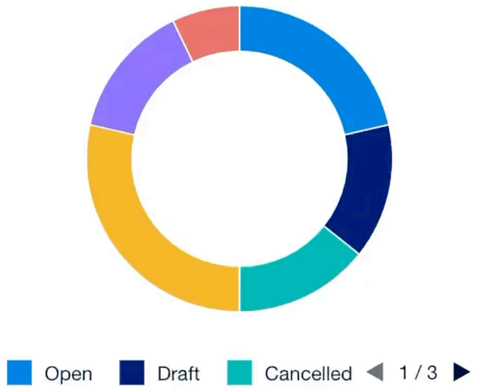
Outstanding
12,195.64

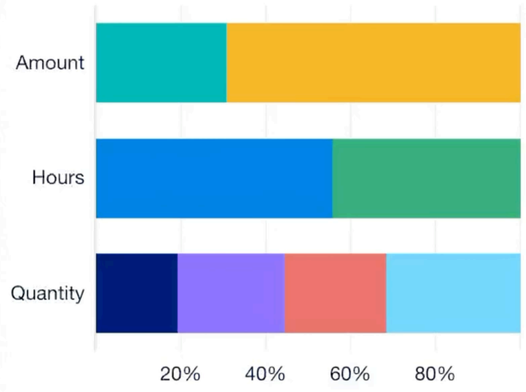
Overdue
10,527.64

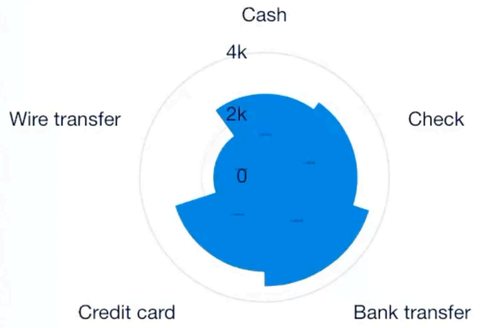
Invoices from
Filter by: Invoice Date
On or After 01 / 07 / 2026

Invoiced and paid by invoice month, with collection rate

Collection rate

Invoice value by status and customer


Billing by unit


Payments by method


Company	Total	Cancelled	Draft	Marked as paid	Marked as refunded	Partially paid
	Invoice Total (sum)	Invoice Total (sum)	Invoice Total (sum)	Invoice Total (sum)	Invoice Total (sum)	Invoice Total (s
Analytical Engines Ltd	5,613.56	N/A	N/A	2,700.00	N/A	2,91

Filters

Compose
Widgets | Page
Search...

Column Chart
Grouped | Stacked | 100% Stacked

Bar Chart
Grouped | Stacked | 100% Stacked

Line Chart
Line

Area Chart
Area | Stacked | 100% Stacked

Value
KPI | Radial Gauge | Linear Gauge

Scatter Chart
Scatter | Bubble

Table
Table | Pivot

Data
Search...

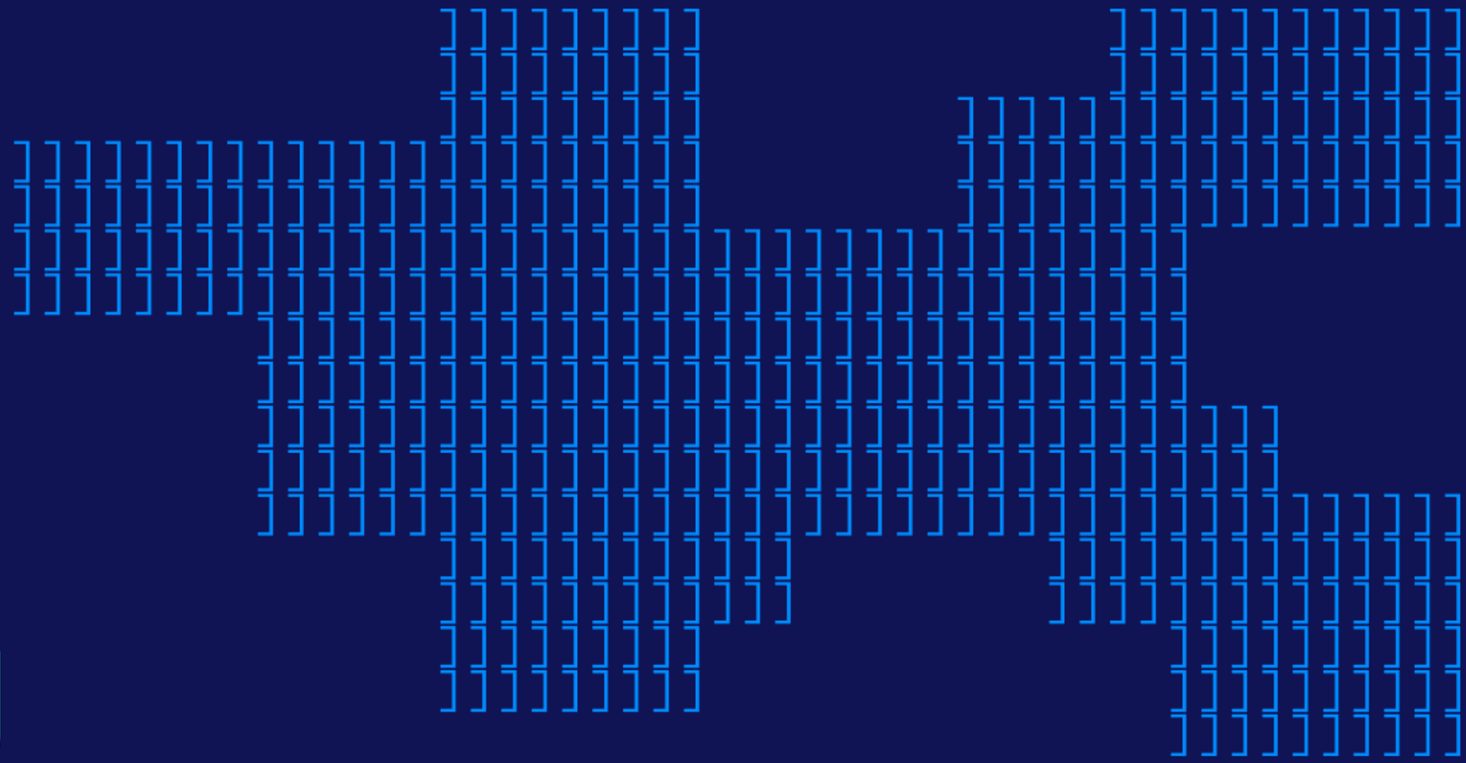
Customers
- Customer
- Company
- Email

Invoices
- Invoice
- Invoice Status
- Status Group
- Currency
- Payment Term
- Invoice Date
- Due Date
- Invoice Total
- Invoiced
- Paid
- Refunded
- Net Collected
- Is Receivable
- Outstanding
- Overdue
- Days Overdue
- Aging Bucket
- Days to Pay
- Tax
- Collection Rate
- Overdue Share
- Refund Rate

Invoice Line Items

06

Connecting Studio to your UI



Changing the mode

Dashboard.ts

```tsx
// Toggle the modes of AG Studio on a button click by changing a single prop

export function Dashboard() {
  const apiRef = useRef<AgStudioApi | null>(null);
  const [mode, setMode] = useState<AgStudioMode>('view');

  return (
    <>
      <button onClick={() => setMode(mode === 'edit' ? 'view' : 'edit')}>
        {mode === 'edit' ? 'Done' : 'Edit'}
      </button>

      <AgStudio
        mode={mode}
        onApiReady={(event) => { apiRef.current = event.api; }}
      />
    </>
  );
}
```

Implementing Undo & Redo

Example.ts

```
// Hook into undo/redo actions via the AG Studio API, and track the changes

export function Dashboard() {
  const apiRef = useRef<AgStudioApi | null>(null);

  return (
    <>
      <button onClick={() => apiRef.current?.undo()}>Undo</button>
      <button onClick={() => apiRef.current?.redo()}>Redo</button>

      <AgStudio
        onApiReady={(event) => {
          apiRef.current = event.api;
        }}
      />
    </>
  );
}
```

Saving & Restoring Reports

Example.ts

```tsx
// Save & load reports by using the getState and setState APIs, triggered by buttons

export function Dashboard({ reports }: { reports: SavedReport[] }) {
  const apiRef = useRef<AgStudioApi | null>(null);

  const save = () => {
    const state = apiRef.current!.getState();
    localStorage.setItem(`report-state`, JSON.stringify(state));
 };

  const load = (report: SavedReport) => {
    const api = apiRef.current!;
    apiRef.current.setState(report.state);
 };

  return (
    <>
      {reports.map((report) => (
        <button key={report.id} onClick={() => load(report)}>{report.name}</button>
      ))}
      <button onClick={save}>Save</button>
      <AgStudio onApiReady={ … } />
    </>
  );
}
```

Changing the Active Page

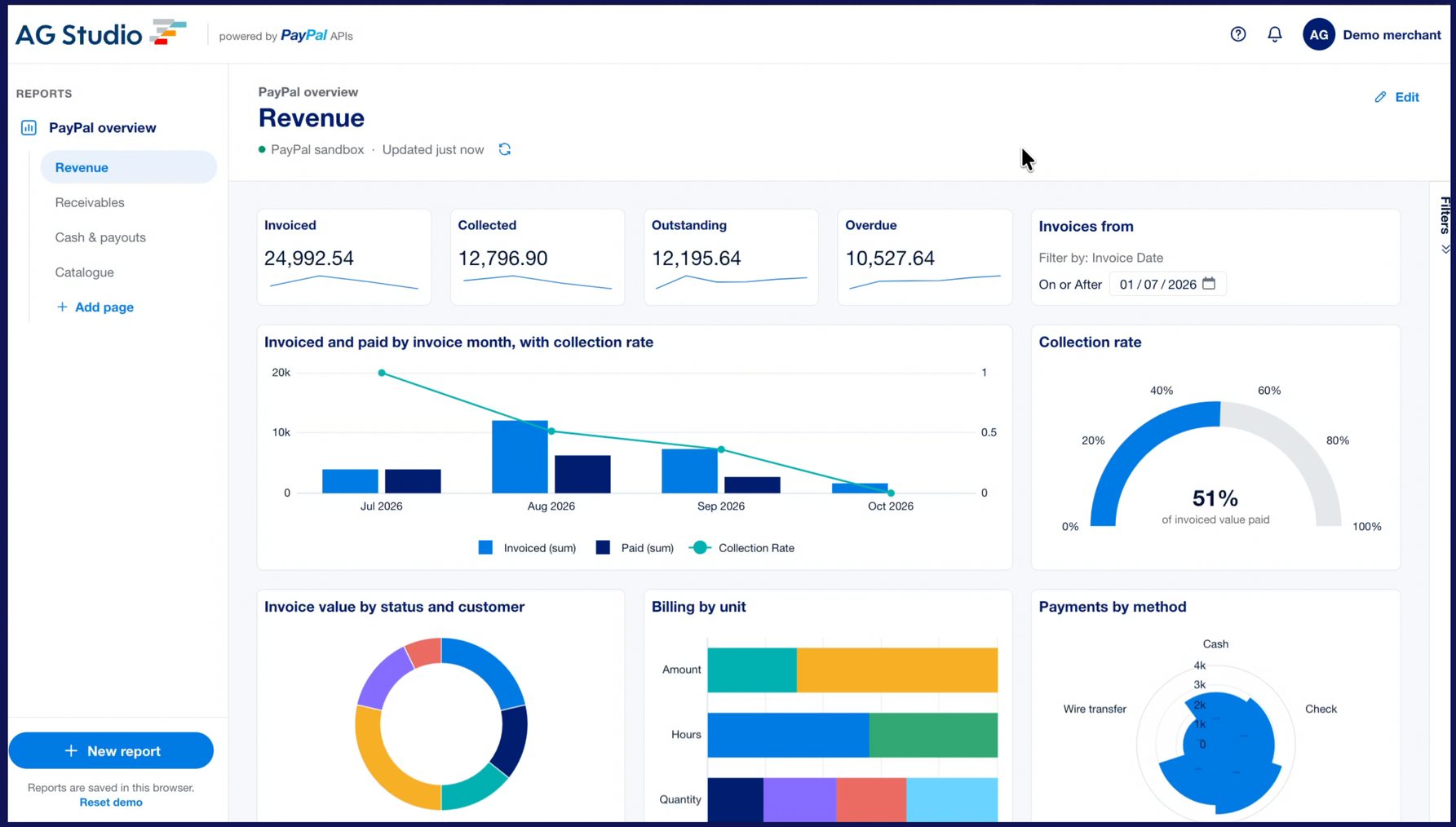
Page-utils.ts

```ts
// Add utility methods for selecting and adding pages by modifying state

export const selectPage = (s: AgReportState, pageId: string): AgReportState => ({
  ...s,
  selectedPageId: pageId,
});

// An empty page is just { id }
export const addPage = (s: AgReportState, pageId: string): AgReportState => ({
  ...s,
  pages: [...s.pages, { id: pageId }],
  selectedPageId: pageId,
});
```

Changing the Active Page

Dashboard.tsx

```tsx
// Apply page state changes via buttons on the UI

import { selectPage, addPage } from 'page-utils';

export function Dashboard() {
  const apiRef = useRef<AgStudioApi | null>(null);
  const [state, setState] = useState<AgReportState>(dashboardReport);

  return (
     <nav>
        {state.pages.map((page, i) => (
          <button key={page.id} onClick={selectPage(s, page.id))}>Page {i + 1}</button>
        ))}
        <button onClick={() => addPage(state, crypto.randomUUID()))}>+ Add page</button>
      </nav>

      <AgStudio
        initialState={dashboardReport}
        onStateUpdated={(event) => setState(event.state)}
      />
 );
}
```



AG Grid × PayPal Developer

REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

+ Add page

PayPal overview

Revenue

PayPal sandbox · Updated just now

Edit

Invoiced
24,992.54

Collected
12,796.90

Outstanding
12,195.64

Overdue
10,527.64

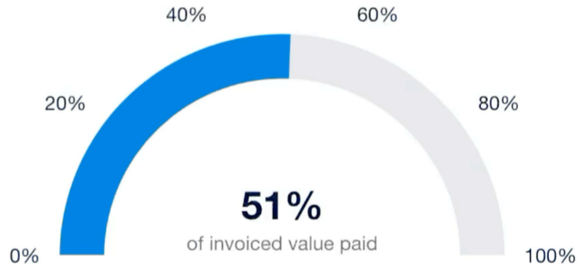
Invoices from

Filter by: Invoice Date

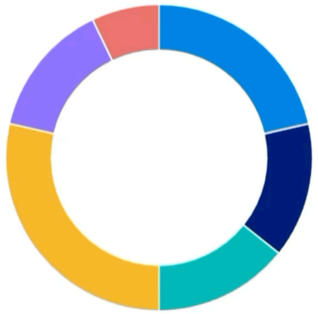
On or After 01 / 07 / 2026

Invoiced and paid by invoice month, with collection rate

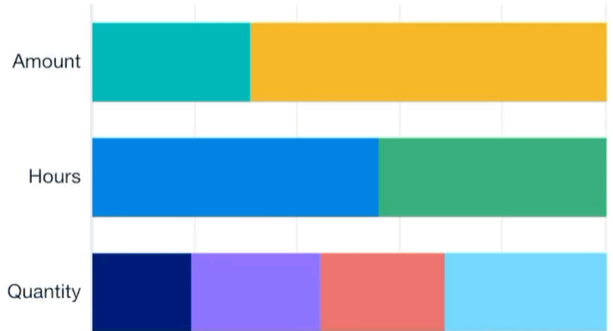
Collection rate



Invoice value by status and customer



Billing by unit



Payments by method

Filters

+ New report

Reports are saved in this browser.
Reset demo

05

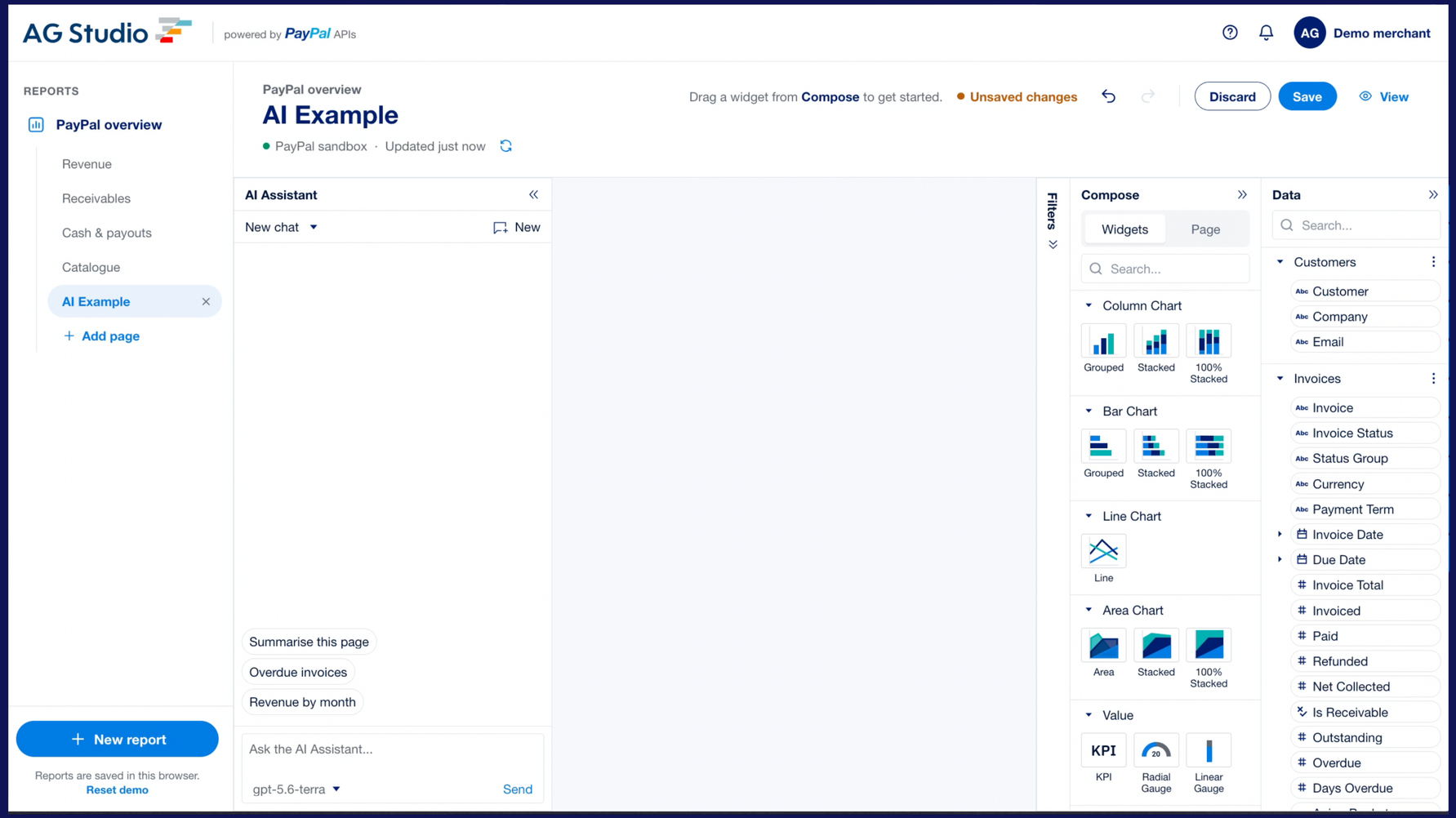
Adding an AI Assistant

Configuring the AI Harness

Configuring the AI Harness

Example.ts

```ts
// Implement your own LLM by passing an adapter to AG Studio

import { openaiAdapter } from '@/app/dashboard-api/components/studio/ai/openai-adapter';

const ai: AgAiHarnessSetup = ({ api }) =>
  createAiHarness(api, {
    adapter: openaiAdapter({ endpoint: '/api/ai', model: 'gpt-5.4-mini' }),
    promptStarters: [
      { label: 'Overdue invoices', prompt: 'Which customers owe the most?' },
      { label: 'Revenue by month', prompt: 'Chart monthly paid invoice revenue' },
    ],
  });

// Dashboard from slides 4 and 8, with two additions: the AI module and the `ai` prop
export function Dashboard({ data, licenseKey }) {
  return (
    <AgStudioProvider licenseKey={licenseKey} modules={[AgStudioAiModule]}>
      <AgStudio ai={ai} /* ... */ />
    </AgStudioProvider>
  );
}
```



AG Grid × PayPal Developer

REPORTS

PayPal overview

Revenue

Receivables

Cash & payouts

Catalogue

AI Example

Add page

PayPal overview

AI Example

PayPal sandbox · Updated just now

Drag a widget from **Compose** to get started. Unsaved changes

Discard Save View

AI Assistant

New chat New

Summarise this page

Overdue invoices

Revenue by month

Ask the AI Assistant...

gpt-5.6-terra Send

Filters

Compose

Widgets Page

Search...

Column Chart

Grouped Stacked 100% Stacked

Bar Chart

Grouped Stacked 100% Stacked

Line Chart

Line

Area Chart

Area Stacked 100% Stacked

Value

KPI Radial Gauge Linear Gauge

Data

Search...

Customers

Customer

Company

Email

Invoices

Invoice

Invoice Status

Status Group

Currency

Payment Term

Invoice Date

Due Date

Invoice Total

Invoiced

Paid

Refunded

Net Collected

Is Receivable

Outstanding

Overdue

Days Overdue

+ New report

Reports are saved in this browser.

Reset demo

Docs

ag-grid.com/studio

AG Grid × PayPal Developer

Source Code

AG Grid × PayPal Developer

Join the Discord

AG Grid × PayPal Developer

@eddiejaoude

Eddie Jaoude

@jamesswintondev

James Swinton-Bland

@sylwiavargas

Sylwia Vargas

AG Grid × PayPal Developer

Thanks!

AG Grid × PayPal Developer